TEMA 2 · ENTORNOS DE DESARROLLO

Entorno integrado de desarrollo (IDE)

Concepto y funciones · Entornos libres y propietarios · Criterios de elección · Uso básico · Visual Studio y .NET

Vas a saber...

01

Qué es un IDE y qué lo compone

Los cinco componentes comunes a todos ellos

02

Qué funciones ofrece

Del editor con coloración al control de versiones

03

Distinguir libres de propietarios

Y conocer los más utilizados de cada tipo

04

Elegir un IDE con criterio

Sistema operativo, lenguaje, framework y disponibilidad

El tema en seis bloques

1	**Concepto de IDE**	Qué es, qué lo compone y de dónde viene
2	**Funciones de un entorno de desarrollo**	Las herramientas que lo forman y lo que permiten hacer
3	**Entornos libres y propietarios**	Los que no se pagan y los que necesitan licencia
4	**Criterios de elección**	Sistema operativo, lenguaje, framework y herramientas
5	**Uso básico de un IDE**	Generar ejecutables y desarrollar de forma colaborativa
6	**Nuestra elección: Visual Studio**	Visual Studio Community 2019, C# y la plataforma .NET

El tema anterior dejó una puerta abierta

En la unidad anterior hablábamos de las fases del proceso de desarrollo de software. Una de ellas era la fase de codificación, en la cual se hacía uso de algún lenguaje de programación para pasar todas las acciones que debía llevar a cabo la aplicación a algún lenguaje que la máquina fuera capaz de entender y ejecutar. También se hizo alusión a herramientas de apoyo al proceso de programación.

En esta unidad vamos a analizar, instalar y ejecutar estas herramientas para entender su acción y efecto

Cómo se empieza

Muchas personas aprenden a programar utilizando un editor de texto simple, compilador y depurador.

Cómo se acaba

Pero la mayoría, finalmente, terminan haciendo uso de algún entorno de desarrollo integrado (IDE) para crear aplicaciones.

¿Qué es un IDE?

Un entorno integrado de desarrollo (IDE, Integrated Development Environment) es un programa informático que tiene el objetivo de asistir al programador en la tarea de diseñar y codificar un software mediante la inclusión de múltiples herramientas destinadas para dicha tarea

Cada IDE tiene unas características y funcionalidades específicas que lo definen. No obstante, todos mantienen unos componentes comunes.



Los cinco componentes comunes a todos los IDE.

De dónde salen los IDE

Años 70

Nacen los primeros entornos de desarrollo integrados.

Años 90

Se popularizan y se convierten en la herramienta habitual.

El objetivo

Ganar fiabilidad y tiempo en los proyectos de software.

Proporcionan al programador una serie de componentes con la misma interfaz gráfica, con la consiguiente comodidad, aumento de eficiencia y reducción de tiempo de codificación.

¿Un IDE sirve para un lenguaje o para varios?

La norma

Normalmente, un IDE está dedicado a un determinado lenguaje de programación.

La tendencia

Las últimas versiones de los IDE tienden a ser compatibles con varios lenguajes mediante la instalación de plugins adicionales.

Son ejemplos de IDE compatibles con varios lenguajes:

Eclipse

NetBeans

Microsoft Visual Studio

De qué está compuesto un entorno de desarrollo

Los entornos de desarrollo están compuestos por una serie de herramientas software de programación, necesarias para la consecución de sus objetivos.

1

Editor de textos

Resalta y colorea la sintaxis

2

Compilador / intérprete

Detecta errores de sintaxis en tiempo real

3

Depurador

Ejecución, traza y puntos de ruptura

4

Generador automático

Componentes visuales y asistentes

5

Interfaz gráfica

Acceso a bibliotecas y plugins

Las cinco herramientas que componen un entorno de desarrollo.

Editor de textos y compilador / intérprete

Editor de textos

- Se resalta y colorea la sintaxis.
- Función de autocompletar código.
- Ayuda y listado de parámetros de funciones y métodos de clase.
- Inserción automática de paréntesis, corchetes, tabulaciones y espaciados.

Compilador / intérprete

- Detección de errores de sintaxis en tiempo real.
- Características de refactorización.

Depurador, generador e interfaz gráfica

Depurador

- Botón de ejecución y traza.
- Puntos de ruptura.
- Seguimiento de variables.
- Opción de depurar en servidores remotos.

Generador automático

- Visualización, creación y manipulación de componentes visuales.
- Un arsenal de asistentes y utilidades de gestión y generación de código.

Interfaz gráfica

- Permite programar en varios lenguajes con un mismo IDE.
- Interfaz agradable que accede a innumerables bibliotecas y plugins, aumentando las opciones de nuestros programas.

Lo que un IDE hace con tu código

Al escribir

- Editor de código: coloración de la sintaxis.
- Auto-completado de código, atributos y métodos de clases.
- Identificación automática de código.
- Detección de errores de sintaxis en tiempo real.
- Refactorización: cambios menores que facilitan la legibilidad sin alterar la funcionalidad.
- Tabulaciones y espaciados automáticos.

Al construir

- Herramientas de concepción visual para crear y manipular componentes visuales.
- Asistentes y utilidades de gestión y generación de código.
- Archivos fuente en unas carpetas y compilados a otras.
- Compilación de proyectos complejos en un solo paso.
- Generador de documentación integrado.

Trabajar en equipo y afinar el entorno

Control de versiones

- Un único almacén de archivos compartido por todos los colaboradores de un proyecto.
- Ante un error, mecanismo de auto-recuperación a un estado anterior estable.
- Soporta cambios de varios usuarios de manera simultánea.

Depuración y ajustes

- Depuración: seguimiento de variables, puntos de ruptura y mensajes de error del intérprete.
- Aumento de funcionalidades a través de la gestión de módulos y plugins.
- Administración de las interfaces de usuario: menús y barras de herramientas.
- Administración de las configuraciones del usuario.

Libres y propietarios

Entornos integrados libres

Son aquellos con licencia de uso público. No hay que pagar por ellos.

Los más conocidos y utilizados son Eclipse y NetBeans, aunque hay bastantes más.

Entornos integrados propietarios

Son aquellos que necesitan licencia. No son free software: hay que pagar por ellos.

El más conocido y utilizado es Microsoft Visual Studio, que usa el framework .NET.

La licencia no es el único criterio de elección, pero sí el que marca el presupuesto

Entornos integrados libres

IDE	Lenguajes que soporta	Sistema operativo
NetBeans	C/C++, Java, JavaScript, PHP, Python	Windows, Linux, Mac OS X
Eclipse	Ada, C/C++, Java, JavaScript, PHP	Windows, Linux, Mac OS X
Gambas	Basic	Linux
Anjuta	C/C++, Python, Javascript	Linux
Geany	C/C++, Java	Windows, Linux, Mac OS X
GNAT Studio	Fortran	Windows, Linux, Mac OS X

Tres de los seis sólo funcionan sobre un sistema operativo. Hoy NetBeans es Apache NetBeans, Anjuta ya no se mantiene y GNAT Studio es un IDE de Ada, no de Fortran.

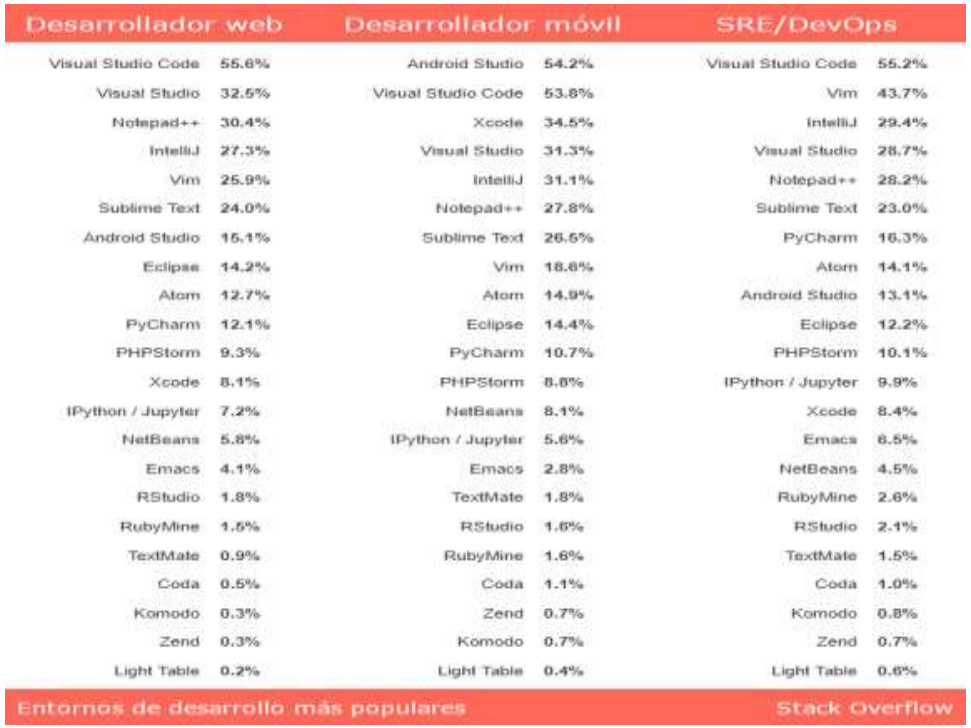
Entornos integrados propietarios

IDE	Lenguajes que soporta	Sistema operativo
Microsoft Visual Studio	Basic, C/C++, C#	Windows
FlashBuilder	ActionScript	Windows, Mac OS X
C++ Builder	C/C++	Windows
Turbo C++ profesional	C/C++	Windows
IntelliJ	Java	Windows, Mac OS X
JCreator	Java	Windows, Linux, Mac OS X
Xcode	C/C++, Java	Mac OS X

Microsoft Visual Studio es el que usaremos. FlashBuilder quedó obsoleto con el fin de Flash (dic. 2020) y Turbo C++ es un producto histórico.

Qué usan los desarrolladores

Según una encuesta realizada por Stack Overflow, la comunidad de desarrolladores, éstos son los editores de código y entornos de desarrollo más populares:



Desarrollador web		Desarrollador móvil		SRE/DevOps	
Visual Studio Code	55.6%	Android Studio	54.2%	Visual Studio Code	55.2%
Visual Studio	32.5%	Visual Studio Code	53.8%	Vim	43.7%
Notepad++	30.4%	Xcode	34.5%	IntelliJ	29.4%
IntelliJ	27.3%	Visual Studio	31.3%	Visual Studio	28.7%
Vim	25.9%	IntelliJ	31.1%	Notepad++	28.2%
Sublime Text	24.0%	Notepad++	27.8%	Sublime Text	23.0%
Android Studio	16.1%	Sublime Text	26.6%	PyCharm	16.3%
Eclipse	14.2%	Vim	18.6%	Atom	14.1%
Atom	12.7%	Atom	14.9%	Android Studio	13.1%
PyCharm	12.1%	Eclipse	14.4%	Eclipse	12.2%
PHPStorm	9.3%	PyCharm	10.7%	PHPStorm	10.1%
Xcode	8.1%	PHPStorm	8.8%	IPython / Jupyter	9.9%
IPython / Jupyter	7.2%	NetBeans	8.1%	Xcode	8.4%
NetBeans	5.8%	IPython / Jupyter	5.6%	Emacs	6.5%
Emacs	4.1%	Emacs	2.8%	NetBeans	4.5%
RStudio	1.8%	TextMate	1.8%	RubyMine	2.6%
RubyMine	1.5%	RStudio	1.6%	RStudio	2.1%
TextMate	0.9%	RubyMine	1.6%	TextMate	1.5%
Coda	0.5%	Coda	1.1%	Coda	1.0%
Komodo	0.3%	Zend	0.7%	Komodo	0.8%
Zend	0.3%	Komodo	0.7%	Zend	0.7%
Light Table	0.2%	Light Table	0.4%	Light Table	0.6%

Entornos de desarrollo más populares — Stack Overflow

Edición antigua de la encuesta. Atom fue archivado en dic. 2022, y Visual Studio Code —que encabeza los tres perfiles— no está en ninguna de las dos tablas anteriores.

Cómo se elige un IDE

Para poder elegir correctamente el IDE con el que vamos a trabajar necesitamos saber las características que buscamos en él, si satisface nuestras necesidades y si nosotros mismos cumplimos los requisitos del IDE, más allá de los requerimientos técnicos de hardware.

1

Sistema operativo

En cuál vamos a trabajar y, más importante, para cuál vamos a desarrollar

2

Lenguaje y framework

Qué lenguajes ofrece cada IDE y sobre qué plataforma de trabajo

3

Herramientas y disponibilidad

Qué funciones tiene y qué precio

Los tres criterios van en orden: el sistema operativo es el más restrictivo.

Sistema operativo

Uno de los criterios más restrictivos es saber en qué sistema operativo vamos a trabajar y, más importante aún, para qué sistema operativo vamos a desarrollar nuestro software

Sobre virtualizar

Aunque hoy en día podemos virtualizar o emular el sistema operativo o IDE que queremos usar, es aconsejable no utilizar esos métodos si no es como último recurso.

La regla práctica

Si estamos desarrollando aplicaciones para Linux, resultaría bastante inusual desarrollar la aplicación en Windows para ello.

Lenguaje de programación y framework

Un IDE puede soportar uno o varios lenguajes de programación, por lo que saber en qué lenguaje vamos a codificar y qué lenguajes nos ofrecen los distintos IDE es una información valiosa. Este criterio va de la mano con el sistema operativo.

Si quisiéramos desarrollar en Visual Basic bajo un sistema operativo Linux, no sería Visual Studio nuestra opción, sino que tendríamos que utilizar Gambas.

Lo mismo ocurre con las plataformas de trabajo, también llamadas framework. Si fuésemos a desarrollar con Visual Basic con la plataforma .NET bajo Linux, tendríamos que usar MonoDevelop en lugar de Visual Studio.

Los dos ejemplos del material: el mismo lenguaje, distinto sistema operativo, distinto IDE.

Herramientas y disponibilidad

Herramientas

- Seguramente encontraremos varios IDE que cumplen los requisitos de lenguaje y sistema operativo, pero no todos tienen las mismas funciones.
- El trabajo colaborativo puede ser más restrictivo: si el equipo usa Team Server Foundation (TFS), el único modo de usarlo es Visual Studio.
- Si usan Subversion (SVN), se puede usar indistintamente NetBeans, Eclipse o IntelliJ IDEA.

Disponibilidad

- El aspecto más restrictivo reside en el precio de la aplicación: según nuestro presupuesto podremos acceder a diferentes IDE.
- Aunque el presupuesto sea escaso existen soluciones muy económicas e incluso gratuitas.
- Tendremos alternativas suficientes que cumplirán al menos la mayoría de los criterios que necesitamos.

TFS, que el material llama «Team Server Foundation», se llama hoy Azure DevOps Server.

El mayor problema no es elegir mal

El mayor problema que nos podemos encontrar no es ya elegir incorrectamente, sino no saber qué funcionalidades podría estar otorgando un IDE u otro

Podríamos no conocer todas las funcionalidades que puede llegar a ofrecer y, por tanto, deberemos invertir una gran cantidad de tiempo investigando y documentándonos a fondo sobre los IDE potenciales que podemos usar, o invertir ese tiempo en probarlos de manera empírica.

No debemos agobiarnos: al igual que con los lenguajes de programación, no podemos pretender empezar sabiendo todas las posibilidades, y tenemos que ir aprendiéndolas de modo incremental en función de nuestra experiencia.

¿Para qué sirve un IDE?

Para desarrollar software

Los entornos de desarrollo integrado son por definición una herramienta de desarrollo de software, por lo que la respuesta resulta bastante obvia. No obstante, la tarea de un IDE no se queda ahí.

Si sólo fuese eso...

Si la tarea de un IDE sólo fuese desarrollar aplicaciones, entonces un editor de texto y un compilador harían la misma función.

La particularidad de un IDE

Reside en la cantidad y calidad de las funcionalidades añadidas que ofrece al desarrollador, por medio de herramientas integradas o externas.

Edición de programas y generación de ejecutables

La necesidad básica que todo IDE debe cubrir es la creación o edición de programas y convertir ese código fuente en código ejecutable.

Sin IDE

Compilaríamos nuestro código fuente y utilizaríamos un enlazador para combinar ese código objeto con nuestras librerías, obteniendo como resultado nuestro programa ejecutable.

Con IDE

Un IDE realiza esa operación de manera conjunta y compacta. Gracias a un gestor de proyectos podemos ajustar las dependencias de cada sección y todas las opciones de compilación.

Los IDE permiten ejecutar de manera virtual el programa que se está codificando en cualquier momento, siempre y cuando no se produzcan errores durante la compilación: así se comprueba la funcionalidad sin publicar ni desplegar la aplicación.

Desarrollo colaborativo

Proceso de desarrollo de un software de manera descentralizada y distribuida, donde los desarrolladores no necesitan conocerse, tener el mismo jefe, ni hablar el mismo idioma, pero trabajan en el mismo proyecto

Dónde se usa

Este modelo es muy habitual en proyectos de software libre, aunque no es restrictivo: cualquier empresa o grupo de desarrollo puede utilizarlo.

Cómo se controla

Para poder llevar un control del código realizado desde tantas y diversas fuentes se utilizan los controles de versiones.

Visual Studio Community 2019

Qué es

Una versión gratuita con todas las características para estudiantes, desarrolladores de código abierto y desarrolladores individuales.

Por qué lo elegimos

Por ser un entorno muy completo, con gran cantidad de características integradas y la solidez del programa en sí mismo.

Es uno de los entornos de desarrollo más pulidos, profesionales y completos del mercado

El lenguaje que utilizaremos para los ejemplos y ejercicios será C#

El material está escrito sobre la edición 2019; hoy existen versiones posteriores a partir de Visual Studio 2022.

¿Qué es .NET?

.NET es una plataforma de desarrollo de código abierto, multiplataforma y gratuita, para crear muchos tipos diferentes de aplicaciones

Con .NET se pueden usar múltiples lenguajes, editores y bibliotecas para construir aplicaciones para la web, el móvil, el escritorio, el IoT (Internet of Things) y para construir juegos.

C#

Lenguaje simple, moderno, orientado a objetos y con seguridad de tipos

F#

Lenguaje funcional que también incluye programación orientada a objetos e imperativa

Visual Basic

Lenguaje con una sintaxis simple y orientado a objetos

C++

Programación orientada a objetos e imperativa

Las tres implementaciones de .NET

.NET Framework

- La implementación de .NET de siempre.
- Compatible con sitios web, servicios y aplicaciones de escritorio, pero sólo en Windows.
- Incluye Windows Forms, WPF y ASP.NET.

.NET Core

- Implementación multiplataforma para sitios web, servidores y consola en Windows, Linux y MacOS.
- Usada por ASP.NET Core y UWP.
- Moderna, eficiente y diseñada para aplicaciones escalables en la nube.
- Código abierto y compatible con .NET Framework.

Xamarin / Mono

- Implementación .NET para ejecutar aplicaciones en los principales sistemas operativos móviles.
- Xamarin es parte del proyecto Mono.
- Biblioteca para desarrollar aplicaciones para iOS y Android.

Esta asignatura se centrará en .NET Framework, porque es la base de desarrollo .NET

ACTUALIZACIÓN · NO PROCEDE DEL MATERIAL

Qué ha cambiado desde que se escribió el tema

El temario se escribió antes de que Microsoft unificara la plataforma. Dos de los tres nombres de la diapositiva anterior ya no se usan.

«.NET Core» ya no se llama así

Desde la versión 5 el nombre es simplemente .NET. Todas las versiones llamadas Core están fuera de soporte. Hoy la versión LTS en activo es .NET 10, mantenida hasta noviembre de 2028.

Xamarin ha sido sustituido

Su relevo es .NET MAUI, que cubre Android, iOS, Mac Catalyst, macOS y tvOS. El soporte de Xamarin terminó el 1 de mayo de 2024.

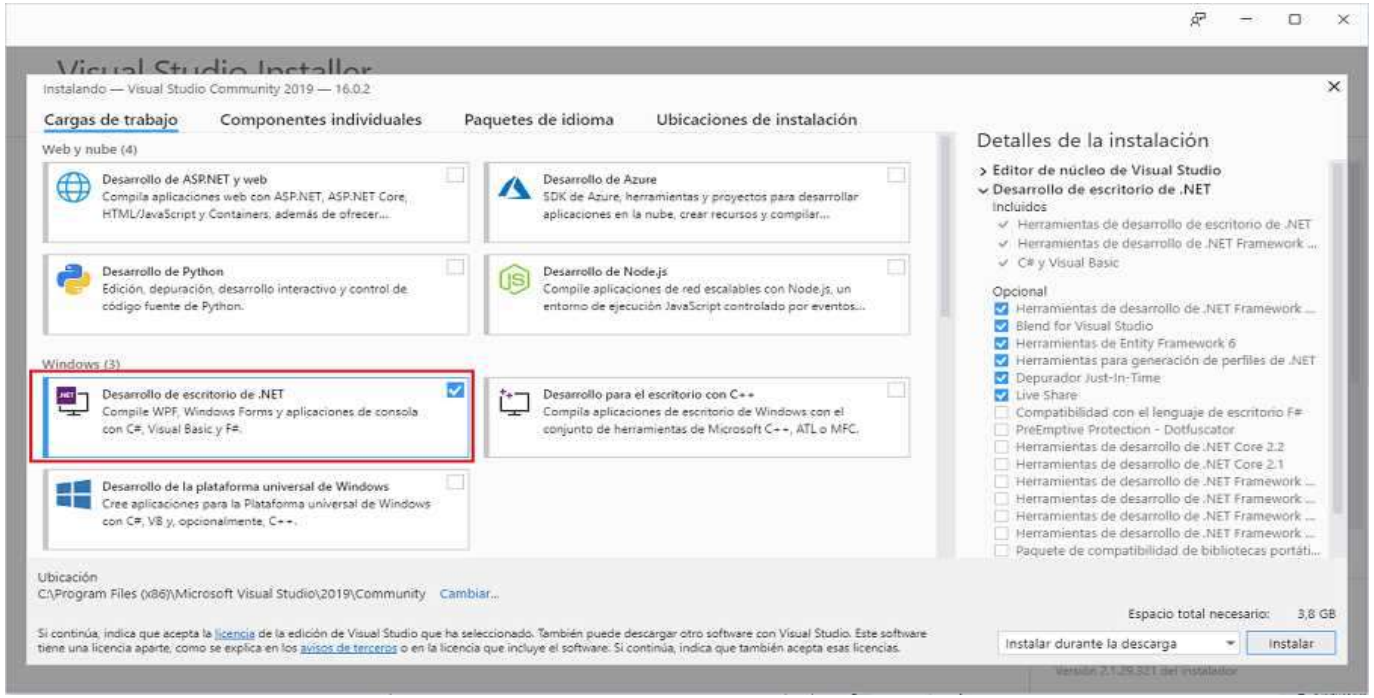
.NET Framework sigue, en mantenimiento

Versión actual 4.8.1. Se distribuye como componente de Windows, así que sigue soportado mientras lo esté el Windows donde se instale. Es la que usaremos en el módulo.

Lo que vas a encontrarte fuera del aula se llama .NET a secas

Instalación

Podemos instalar Visual Studio Community 2019 desde visualstudio.microsoft.com/es/. Es una instalación modular: eliges los módulos según el tipo de aplicaciones que vas a desarrollar.



!

El módulo que hay que marcar

Para desarrollar aplicaciones con interfaz gráfica (aplicaciones de ventanas) para Windows de escritorio, hay que elegir de la sección Windows el módulo «Desarrollo de escritorio de .NET».

Compila WPF, Windows Forms y aplicaciones de consola con C#, Visual Basic y F#.

EN UNA DIAPOSITIVA

Lo que hay que retener

1. Un IDE asiste al programador en diseñar y codificar software reuniendo múltiples herramientas.
2. Todos comparten cinco componentes: editor, compilador, intérprete, depurador y constructor de interfaz.
3. Nacen en los 70 y se popularizan en los 90 para ganar fiabilidad y tiempo.
4. Normalmente sirven para un lenguaje, pero los plugins los abren a varios.
5. Libres: Eclipse y NetBeans los más conocidos. Propietarios: Microsoft Visual Studio.
6. Se elige por sistema operativo, lenguaje y framework, herramientas y disponibilidad.
7. El criterio más restrictivo es para qué sistema operativo vamos a desarrollar.
8. Un IDE compila y enlaza en un solo paso, y permite ejecutar de forma virtual mientras se codifica.
9. El desarrollo colaborativo se controla con los sistemas de control de versiones.
10. Usaremos Visual Studio Community 2019, C# y .NET Framework; fuera del aula, .NET Core se llama hoy .NET y Xamarin es .NET MAUI.